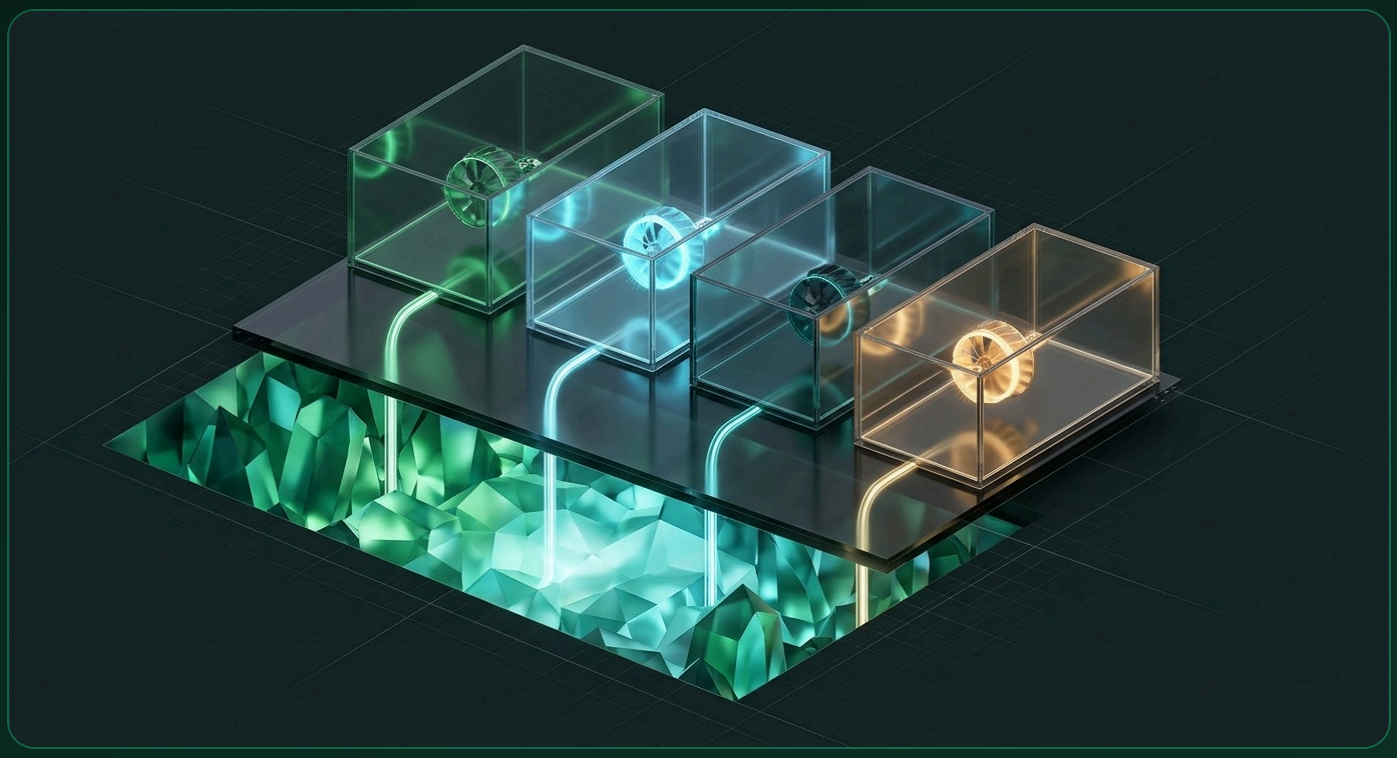
AI AGENT ENGINEERING SERIES • V1.0 EDITION

Mastering Git Worktrees & Agent Safety

The practical visual field guide to running parallel AI coding agents at full speed without file overwrites, port clashes, corrupted lockfiles, or rogue Git commands.

Shared `.git` Plumbing | 1-Agent-1-Branch Discipline | Env, Port & DB Isolation | `PreToolUse` Safety Hooks | Serial Human + CI Merge Gate | Claude, Antigravity, Orca & Cursor



Zero File Collisions
Give every parallel agent its own sibling working directory, index, and branch backed by a single shared `.git` store.

Full Runtime Separation
Isolate untracked state—`node_modules`, `.env.local`, deterministic ports `3001..3009`, and test databases.

Defense-in-Depth Guardrails
Layer path-fencing `PreToolUse` hooks, `pre-commit` door inspectors, OS sandboxes, and a serial PR merge gate.

Why Parallel Agents Collide in a Single Directory

Git protects committed history inside `.git`, but gives your on-disk **working tree** zero concurrency protection. Pointing two autonomous coding agents at one folder turns shared mutable files into silent data loss before a commit ever exists.

Quick Navigation

1. **The 5 Agent Collision Patterns** p. 2
2. **Worktrees vs. Clones vs. Containers** p. 3
3. **Annotated Fleet Workspace Tour** p. 4
4. **Git Worktrees End-to-End Lifecycle** p. 5
5. **Isolation Beyond Git** (Env, Ports, DB) p. 6
6. **Troubleshooting & Pre-Flight Check** p. 12
7. **Safety Hook & Bootstrap Blueprint** p. 7
8. **Serial Merge Gate & Clean Teardown** p. 8
9. **5 Copy-Paste Fleet Recipes** p. 9
10. **Native Support** (Claude, AGY, Orca) p. 10
11. **6-Stage End-to-End Fleet Pipeline** p. 11

The 5 Real-World Parallel Collision Patterns

1 Dirty Index Theft & Silent Overwrites

Agent A edits `checkout.ts` and stages it; 20 seconds later Agent B writes a stale edit plan over the same file—erasing A's work before any commit records it.

2 Branch Switching Mid-Test

While Agent A runs a 90-second test suite, Agent B runs `git checkout -b hotfix`. Both agents now fight over one `HEAD`, staging index, and half-rebased tree.

3 Lockfile & Dependency Clobber

Concurrent `npm install` or shared Python `.venv` editable installs corrupt `node_modules` and generate 5,000-line lockfile conflicts.

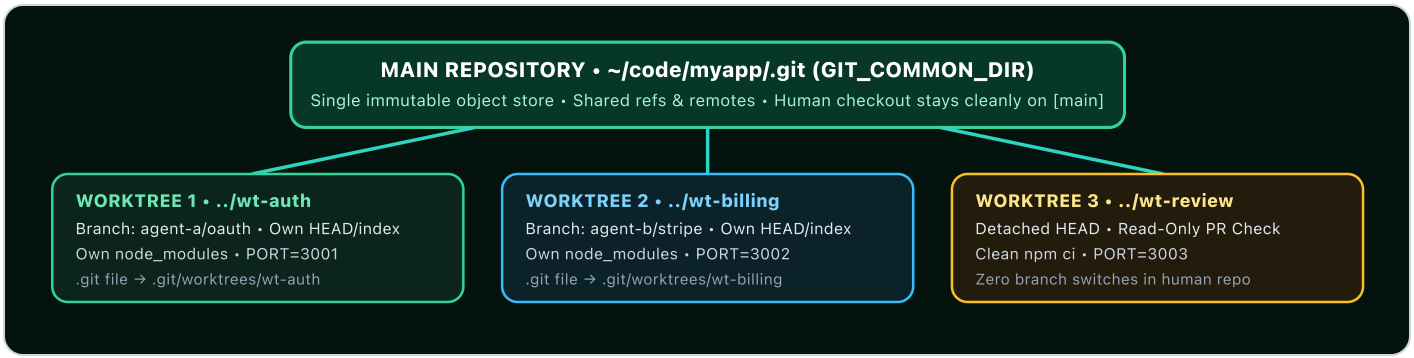
4 Port & Test Database Clashes

Two dev servers crash with `EADDRINUSE` on port `3000`, or two integration suites truncate the same `users` table in a shared local database.

5. Blind Cleanup Commands (`git checkout -- .` & Feedback Repair Loops)

When Agent A sees mysterious test failures caused by Agent B's half-saved edits, A runs `git checkout -- .` or widens types to `any` to force tests green—destroying sibling work in an expensive token-burning loop.

Architecture: One Shared `.git` Warehouse, Isolated Agent Workshops



Choosing the Right Isolation Layer for Your Fleet

Not every parallel task needs a full virtual machine—and "just clone the repo five times" creates silent fetch drift and gigabytes of wasted history. Use this matrix to match isolation weight to task risk.

Isolation Strategy Decision Matrix

Mechanism	Spawn Speed	Disk Footprint	Runtime & Security Boundary	Best Fit
`git worktree` **Recommended Default**	**0.2–2 sec checkout** (+ fast hardlinked `pnpm` install)	**Minimal** 1 shared `.git` object store; working files only	Separate files, index & branch; pair w/ per-folder `PORT`, DB & OS sandbox	Daily 2–6 agent parallel feature builds, bugfixes, refactors & detached PR reviews
Full `git clone` Separate Local Repo	**15–120 sec** Full network or local history copy	**Heavy (N × repo)** 5 agents on a 2 GB repo = 10+ GB duplicated	Separate `.git` config/remotes, but **no automatic branch-lock** across clones	Testing destructive Git history rewrites (`filter-repo`) or alternate remote URLs
Worktree + Container Docker / Devcontainer	**3–15 sec** Cached base image + bind mount	**Moderate** Shared Docker layers + per-worktree volume	**Kernel namespace isolation**: hides `~/.ssh`, `~/.aws`, ports & host filesystem	Unattended overnight runs, auditing untrusted packages, or running risky migrations
Cloud Sandbox VM Actions VM / MicroVM	**10–45 sec** Ephemeral cloud image boot	**Zero local disk** Runs entirely off your laptop	**Hard hardware/VM boundary** + egress firewall & repo-scoped token	Async background fleets (20+ tickets), heavy CI test sharding, or mobile dispatch

Why "Just Give Each Agent a `git clone`" Backfires

1 Fetch Drift
Running `git fetch` in Clone A leaves Clones B and C building on stale 3-day-old `main` commits, creating avoidable merge conflicts.

2 Blind Branch Clashes
Two separate clones happily check out the exact same branch name locally and only collide when both try to push.

3 Scattered Visibility
`git worktree list` inventories every active agent workshop in one command; scattered clones hide orphaned work across folders.

4-Point Fleet Readiness Prerequisites (Verify Once)

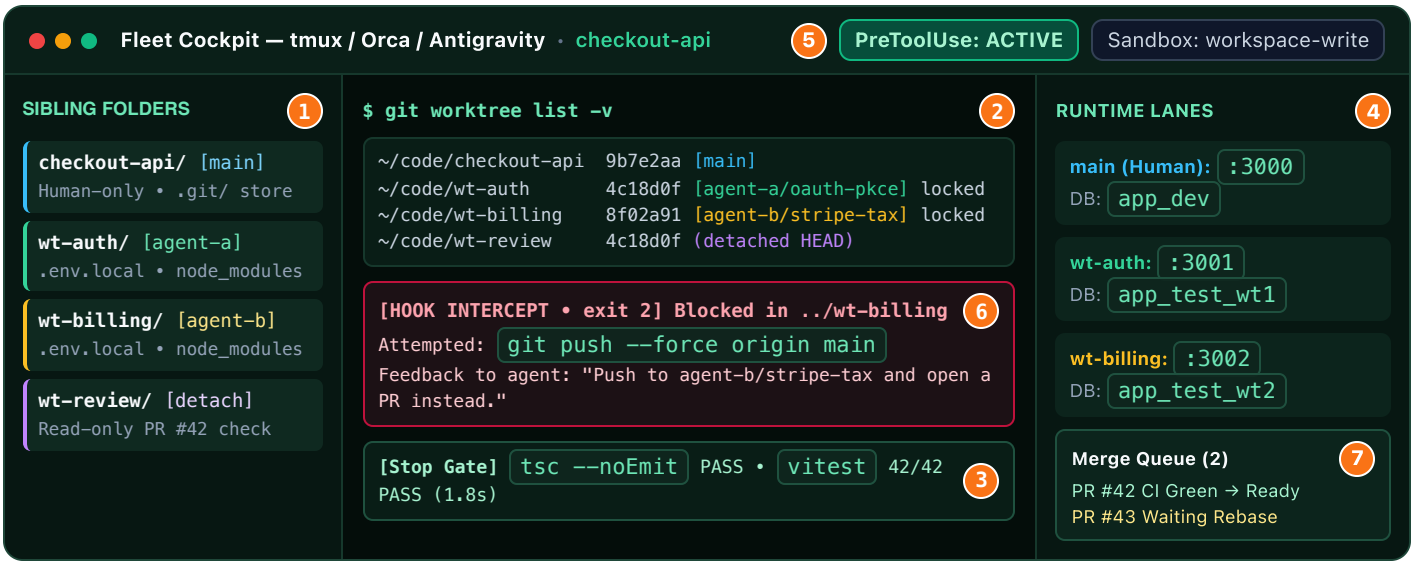
1 Git 2.30+ & Clean Tooling
Verify `git --version` (supports `lock`, `move`, `repair`, and `--porcelain`), plus `jq` for parsing JSON hook payloads and `gh auth status` for PR gates.

2 Fast Deterministic Checks
Guardrails can only enforce checks that exist. Ensure `lint`, `typecheck`, and unit `test` scripts run cleanly and exit non-zero on failure.

Golden Precondition: Never Launch a Fleet From a Red `main`
Always run `git pull --ff-only && npm test` in your human checkout *before* spawning worktrees. Launching four agents from a broken baseline guarantees all four will burn their first hour diagnosing and divergently "fixing" a bug none of them caused!

Anatomy of a Well-Organized Multi-Agent Fleet

Keep linked worktrees as **sibling folders** beside your main repository (`../wt-auth`, `../wt-billing`)—or inside a gitignored `.worktrees/` directory—so file watchers, bundlers, and search tools never index duplicate trees.



Numbered Fleet Cockpit Reference

1 **Sibling Worktree Layout:** Placing `../wt-auth` next to `checkout-api/` prevents nested `tsconfig.json`, ESLint, or bundler crawl issues.

2 **Live Fleet Inventory (`git worktree list -v`):** Shows every directory, exact commit SHA, checked-out branch, and active `locked` ownership flag.

3 **Automated `Stop` Gate:** Blocks an agent from declaring victory until `tsc --noEmit` and fast unit tests actually pass inside its worktree.

4 **Deterministic Port & DB Lanes:** Each workshop receives a dedicated port (`3001`, `3002`) and isolated database schema so live servers never clash.

5 **Active Sandbox & Hook Banner:** Confirms OS filesystem confinement (`.git` & `~/.ssh` protected) and repo-committed hook enforcement.

6 **`PreToolUse` Real-Time Intercept:** Catches destructive shell commands *before execution* via exit code `2` and teaches the agent the safe path.

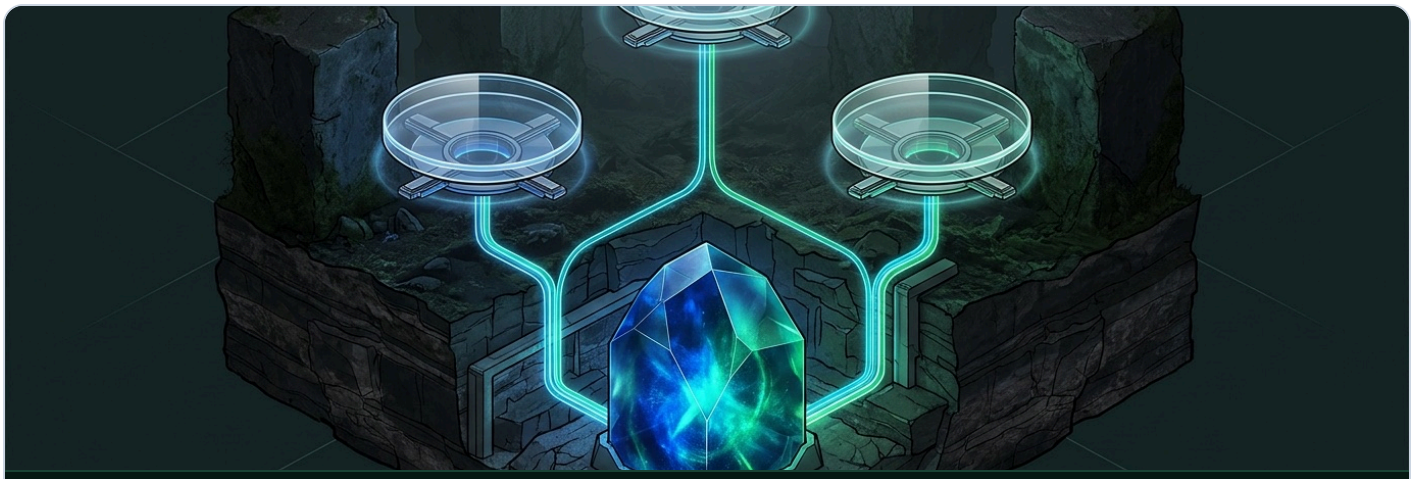
7 **Serial Merge Gate Queue:** Human checkout stays cleanly on `main` while completed agent branches land one PR at a time through CI.

Folder Placement Rule: Sibling Directories Beat Unignored Subfolders

Prefer sibling paths like `../wt-auth`. If your harness stores worktrees inside the repo (such as `.worktrees/` or `.claude/worktrees/`), verify `git check-ignore -q .worktrees` passes *before* spawning so you never accidentally commit an entire nested checkout!

Git Worktrees End-to-End: Plumbing & Daily Commands

When you run `git worktree add`, Git does not duplicate your repository history. Instead, the new folder gets a tiny 1-line `.git` pointer file linking back to `.git/worktrees/<name>/` for its private `HEAD` and staging index.



SHARED (`GIT_COMMON_DIR`): Commits, blobs, branches, tags, remotes & 1 fetch for all.

PRIVATE (`GIT_DIR`): Per-worktree `HEAD`, staging index, and working files on disk.

VS. CLONES (BACKGROUND): Zero multi-GB object duplication or stale-base fetch drift.

The Complete Worktree Command Lifecycle

```
# 1. Spawn a sibling worktree on a brand-new branch explicitly rooted at main:
git worktree add ../wt-auth -b agent-a/oauth-pkce main
# 2. Stamp ownership & prevent accidental pruning (survives reboots):
git worktree lock ../wt-auth --reason "agent-a / PROJ-201 / OAuth PKCE"
# 3. Inspect all active workshops (add --porcelain for scripts, or check plumbing):
git worktree list -v  &&  git -C ../wt-auth rev-parse --git-dir --git-common-dir
# 4. Relocate safely (NEVER use raw 'mv', which breaks the .git pointer contract):
git worktree move ../wt-auth ../wt-auth-v2   # (or 'git worktree repair' if already moved)
# 5. After PR merges: unlock, remove directory, delete merged branch & prune stale metadata:
git worktree unlock ../wt-auth && git worktree remove ../wt-auth
git branch -d agent-a/oauth-pkce && git worktree prune -v
```

A Naming Scheme That Scales

Encode owner + task in both places: folder `../<repo>-<agent>-<slug>` (or `../wt-<slug>`) and branch `<agent>/<slug>`. Keep a human-only checkout permanently parked on `main`.

B Safe Order of Demolition

Always **merge (or abandon) first →** `worktree remove` **second →** `branch -d` **last**. Deleting a branch before removing its worktree orphans commits!

The Iron Rule: Why Git Refuses Double Branch Checkouts

Try checking out `main` in a second worktree and Git halts immediately: `fatal: 'main' is already checked out`. **Celebrate that error!** Git natively enforces the *1-Agent-1-Branch* rule so two worktrees can never silently advance the same branch tip behind each other's backs. Never pass `-f` to override it; for read-only inspection, pass `--detach` instead.

Isolation Beyond Git: What Worktrees Do *Not* Copy

A fresh `git worktree add` checks out **tracked source files only**. Everything gitignored—dependencies, secret `.env` files, build caches, ports, and test databases—must be provisioned per workshop or agents will still collide at runtime.

The 4 Non-Git State Lanes Every Workshop Needs

1 Per-Worktree Dependencies

Never share a parent `node_modules` or shared Python virtualenv across worktrees:

- **Fastest JS/TS:** `pnpm install --frozen-lockfile --prefer-offline` hardlinks from one global content store in ~2 seconds!
- **npm projects:** Use `npm ci` (never bare `npm install`, which mutates `package-lock.json`).
- **Python trap:** Create a fresh `python3 -m venv .venv` inside *each* worktree—an editable `pip install -e .` in a shared venv points imports at whichever folder installed last!

2 Explicit `.env.local` Provisioning

Do not let agents symlink `../myapp/.env` (one edit mutates every sibling):

- **Copy, then override:** Copy `.env.local` into the new worktree during bootstrap, then append workshop-specific overrides (`PORT`, `DATABASE_URL`, `NEXT_PUBLIC_APP_URL`).
- **macOS bonus:** On APFS drives, `cp -c` performs instant copy-on-write cloning even for large local fixture folders.

3 Deterministic Port Offsets (`3001..3009`)

Reserve `:3000` for your human `main` checkout and assign each agent slot a fixed offset:

- Slot `1` → App `PORT=3001`, API `4001`, Vite HMR `24679`.
- Slot `2` → App `PORT=3002`, API `4002`, Vite HMR `24680`.
- Write `PORT` directly into the worktree's `.env.local` so every terminal command inherits it automatically.

4 Isolated Test DBs & `TMPDIR`

Parallel integration tests sharing one DB cause phantom flakes at midnight:

- **SQLite:** Store at `./.tmp/test.db` inside the worktree—instant zero-config isolation!
- **Postgres:** Point each worktree at its own DB or schema (`postgres://localhost/app_test_wt1`) and run migrations on bootstrap.
- **Temp caches:** Export `TMPDIR="$PWD/.tmp"` so test runners never clobber `/tmp` sockets.

Quick Audit: Tracked vs. Untracked State on `git worktree add`

Resource	Included Automatically?	Safe Fleet Bootstrap Rule
`src/`, `AGENTS.md`, `.claude/settings.json`	**Yes** (Tracked in Git)	Commit rules, skills, and hook configs to `main` so every worktree inherits them automatically.
`node_modules/`, `.venv/`, `.husky/_/`	**No** (Gitignored)	Run `pnpm install --frozen-lockfile` / `npm ci` (also regenerates Husky v9 shims!).
`.env.local`, `PORT`, Test DB	**No** (Local runtime)	Generate via a 1-command bootstrap script assigning unique `PORT` + `DATABASE_URL` per slot.

Turning Norms Into Physics: Hooks & Path Fences

Polite prompt instructions eventually lose to a confused model at step 60. Guard every workshop at **two distinct doorways**: an agent-native `PreToolUse` hook that blocks dangerous commands *before execution*, and a Git `pre-commit` hook that blocks broken code *before packaging*.

Color-Coded Anatomy of a Fleet Safety Hook & Bootstrap Stack

[1. BOOTSTRAP ENV] Spawn `../wt-$SLUG` from green `main`, copy `.env.local`, set `PORT=$((3000+SLOT))`, run `pnpm i --prefer-offline`.

[2. SESSION CONTEXT] On `SessionStart`, inject exact `$PWD`, assigned branch name, and assigned port/DB into agent memory.

[3. PATH FENCE] In `PreToolUse` (`Edit|Write|Bash`), resolve target paths and `exit 2` if anything escapes `$WORKTREE_ROOT`.

[4. GIT BLOCKER] Intercept `git reset --hard`, `push --force`, `checkout main`, `clean -fd`, and `--no-verify` with `exit 2`.

[5. PROVE-IT GATE] Wire `.githooks/pre-commit` (lint + `tsc --noEmit` + `.env` secret block) + `Stop` hook running fast tests.

Drop-In `PreToolUse` Path-Fence & Destructive-Git Guard (`.claude/hooks/fleet-guard.sh`)

```
#!/usr/bin/env bash # Commit + chmod +x; wire to PreToolUse matcher "Bash|Edit|Write" in .claude/settings.json
set -euo pipefail
payload=$(cat); root=$(git rev-parse --show-toplevel)
tool=$(jq -r '.tool_name // ""' <<<"$payload")
cmd=$(jq -r '.tool_input.command // ""' <<<"$payload")
f=$(jq -r '.tool_input.file_path // ""' <<<"$payload")
deny() { echo "BLOCKED BY FLEET POLICY: $1" >&2; exit 2; } # CRITICAL: exit 2 blocks; exit 1 only logs!

# A) Path Fence: forbid editing any sibling worktree, parent checkout, or home credential file
if [[ -n "$f" ]]; then
  abs=$(python3 -c 'import os,sys; print(os.path.realpath(sys.argv[1]))' "$f")
  [[ "$abs" == "$root"/* ]] || deny "File '$abs' is outside your worktree ($root). Stay in your lane."
fi
# B) Destructive Git & Cross-Directory Command Blocker
case "$cmd" in
  *"reset --hard"*|*"checkout -- ."*|*"clean -f"*) deny "Destructive discard blocked. Commit or stash instead." ;;
  *"push --force"*|*"push -f "*|*"push origin main"*) deny "Never force-push or push main. Push your branch & open
a PR." ;;
  *"checkout main"*|*"switch main"*|*"--no-verify"*) deny "Stay on your worktree branch and keep Git hooks
enabled." ;;
  *"../"*|*"rm -rf /"*|*"rm -rf ~"*) deny "Cross-directory '..' traversal and root recursive deletes are
forbidden." ;;
esac; exit 0
```

Watch Out for the Exit-Code `1` Trap!

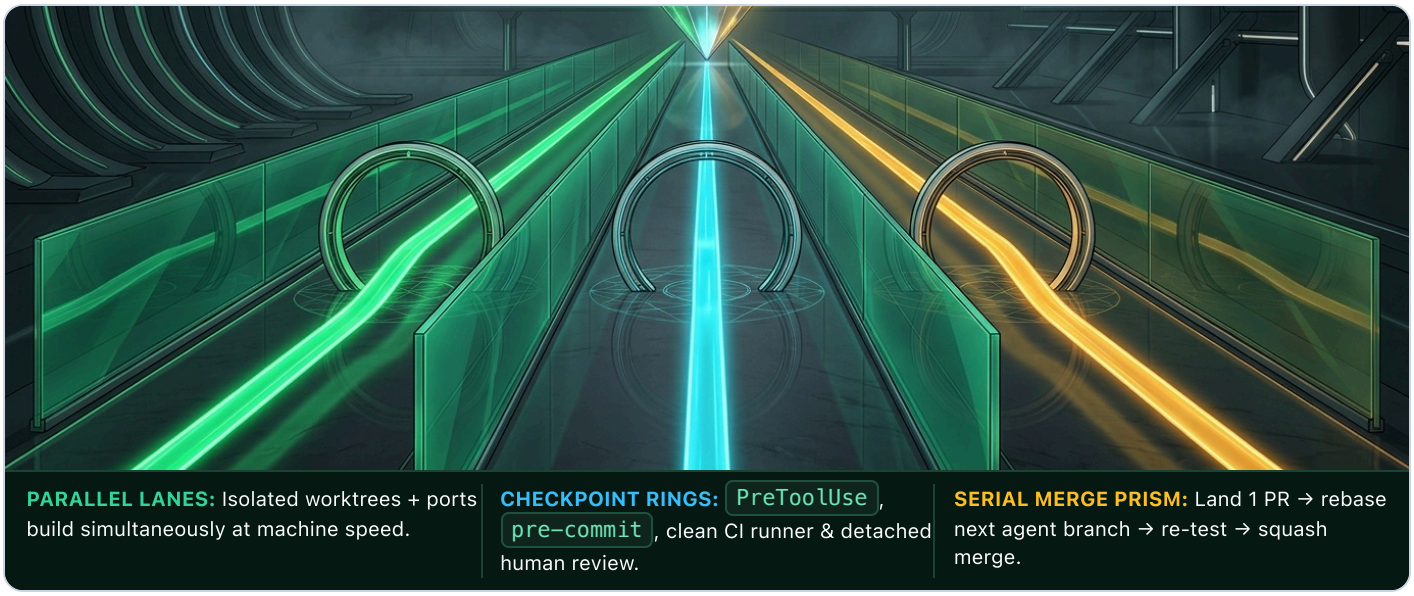
In Claude Code hooks, `exit 1` is treated as a *non-blocking script error* and the dangerous command still runs! Always return `exit 2` (with clear corrective instructions on `stderr`) to hard-block execution.

Test Your Hook in 5 Seconds Without an Agent

Pipe a mock JSON payload straight in your shell: `echo '{"tool_input":{"command":"git reset --hard"}}' | .claude/hooks/fleet-guard.sh; echo $?` → must print `2`!

Parallel Development, Serial Integration

The secret to landing five parallel agent branches without breaking `main` is simple: **build in parallel, merge serially one PR at a time.** Every remaining branch rebases onto the newly updated `main` inside its own workshop before advancing.



PARALLEL LANES: Isolated worktrees + ports build simultaneously at machine speed.

CHECKPOINT RINGS: `PreToolUse`, `pre-commit`, clean CI runner & detached human review.

SERIAL MERGE PRISM: Land 1 PR → rebase next agent branch → re-test → squash merge.

The 4-Step Serial Merge & Teardown Protocol

1 Review via `--detach` Worktree

Never switch branches in your human checkout. Run `git worktree add ../wt-review --detach origin/agent-a/oauth`, run `pnpm i && pnpm test`, and audit test diffs for softened assertions or widened `any` types.

2 Squash-Merge PR #1 Through CI

Enable GitHub **Require branches to be up to date** + `CODEOWNERS` on `.claude/` and `.githooks/`. Run `gh pr merge 41 --squash --delete-branch` to collapse noisy micro-commits into one clean commit.

3 Rebase Queued Agents In-Place

Inside `../wt-billing`, run `git fetch origin && git rebase origin/main`. If shared barrel exports or routes conflict, ask **that workshop's owning agent** to reconcile its change against upstream `main`, then re-run tests!

4 Demolish & Prune Cleanly

Pull fresh `main` locally, then retire merged workshops: `git worktree unlock ../wt-auth && git worktree remove ../wt-auth`, `git branch -d agent-a/oauth`, and `git worktree prune -n` (dry run) → `-v`.

Operator Superpower: Control Any Workshop From Your Main Seat With `git -C`

Stay parked in your human `main` folder and inspect or push any sibling workshop without `cd`-ing back and forth: `git -C ../wt-auth log --oneline main..HEAD` or `git -C ../wt-auth status -s`.

5 Copy-Paste Worktree Recipes for Daily Engineering

Keep these five battle-tested patterns handy whenever you want to parallelize a multi-slice roadmap, compare competing designs side by side, or ship an urgent hotfix without disturbing a running agent.

Scenario	Exact Commands / Script Pattern	Why It Works Safely

1. 3-Slice Parallel Feature Build
DB + API + UI in parallel
	`git worktree add ../wt-schema -b ag1/schema main`
	`git worktree add ../wt-api -b ag2/api-routes main`
	`git worktree add ../wt-ui -b ag3/react-views main`
	Assign disjoint directories per prompt; merge `schema` → `api` → `ui`.
		Scoping each agent to disjoint folders eliminates 90% of semantic merge conflicts up front while multiplying throughput 3x.

2. Competing Architecture Bake-Off
Race 2 designs on 1 spec
	`git worktree add ../wt-redis -b exp/redis-cache main`
	`git worktree add ../wt-sqlite -b exp/sqlite-lru main`
	Give both agents the exact same benchmark test; run servers on `:3001` & `:3002`; merge winner, `remove` + `branch -D` loser.
		Lets you evaluate real latency, code clarity, and bundle size across two live implementations before committing to one.

3. Urgent Prod Hotfix Mid-Run
Zero stashing or pausing
	Leave overnight agent humming in `../wt-refactor`.
	`git fetch origin`
	`git worktree add ../wt-hotfix -b fix/null-session origin/main`
	Patch, test on `:3009`, push PR, merge, then `git worktree remove ../wt-hotfix`.
		You never touch `git stash` or interrupt a 45-minute autonomous refactor; the overnight agent simply rebases later.

4. Read-Only PR & Security Audit
Zero branch claim
	`git fetch origin pull/108/head:pr-108`
	`git worktree add ../wt-audit --detach pr-108`
	`cd ../wt-audit && pnpm i --frozen-lockfile && codex --sandbox read-only`
		`--detach` + `read-only` OS sandbox lets a reviewer agent inspect and test untrusted PR code without touching your checkout.

5. One-Command Fleet Bootstrapper
`scripts/wt-spawn.sh`
	`SLOT=$1; SLUG=$2; DIR="../wt-${SLUG}"; PORT=$((3000+SLOT))`
	`git worktree add "$DIR" -b "agent-${SLOT}/${SLUG}" main`
	`cp .env.local "$DIR/.env.local" && echo "PORT=$PORT" >> "$DIR/.env.local"`
	`(cd "$DIR" && pnpm i --frozen-lockfile --prefer-offline)`
	`git worktree lock "$DIR" --reason "slot-$SLOT / $SLUG"`
		Reduces workshop creation—branch, lock, secrets, unique port, and dependencies—to one repeatable 4-second command: `./scripts/wt-spawn.sh 1 auth`.

Weekly Housekeeping One-Liner: Spot Forgotten Orphan Worktrees

Run `git worktree list --porcelain` (or compare against a `.fleet/registry.tsv` log) every Friday afternoon. Any unlocked worktree whose branch already merged into `main` can be reclaimed in seconds—keeping disk space and mental overhead crystal clear.

Native Worktree Flags Across Modern Agent Harnesses

Before running manual `git worktree add` commands, check whether your coding harness manages worktrees natively. Built-in flags automatically place folders, track session state, and clean up idle trees—**never fight your harness!**

Harness-by-Harness Worktree & Sandbox Reference

Tool / Harness	Native Worktree Command / Config	Built-In Guardrails & Key Gotchas
Claude Code Anthropic CLI & IDE	`claude --worktree feat-auth` (`-w`) `claude -w feat-auth --tmux` In-session tool: `EnterWorktree`	Creates isolated worktrees under `.claude/worktrees/`; supports `WorktreeCreate` / `PreToolUse` / `Stop` hooks in committed `.claude/settings.json` + macOS Seatbelt / Linux bubblewrap `/sandbox`.
Google Antigravity App, IDE & `agy` CLI	Subagents: `Workspace: "share"` (worktree) Swarms (`/teamwork`): `workspace: "share"` Full clone mode: `workspace: "branch"`	`"share"` gives each parallel author its own working copy sharing `.git`; read-only researchers keep `"inherit"`. `proceed-in-sandbox` + Non-Workspace File Access `deny` fences paths.
Orca Multi-Agent Orchestrator	`orca worktree create` · `orca worktree list` `orca worktree ps` · `orca worktree rm`	Manages child worktrees, dedicated PTY terminals, sleeping recipe VMs, and cross-agent handoffs with live status tracking via `orca worktree ps --json`.
Cursor Parallel Background/Local	Parallel Agent Worktrees UI + `.cursor/worktrees.json` (setup steps)	Automates dependency install & `.env` copy per spawned worktree. Commit `.cursor/permissions.json` with strict **Allowlist** mode (prefix-matched; deny wins).

Codex CLI & aider
Terminal Agents
	Pair w/ `wt-spawn.sh`, then launch:
	`codex --sandbox workspace-write`
	`aider --git-commit-verify --auto-test`
		Codex's `workspace-write` keeps `.git` **read-only** at the OS kernel level! In aider, **always pass `--git-commit-verify`** (its default skips `pre-commit` hooks).

Detect Before Creating + The Truth About Permission-Bypass Flags

1 Auto-Detect Existing Worktrees

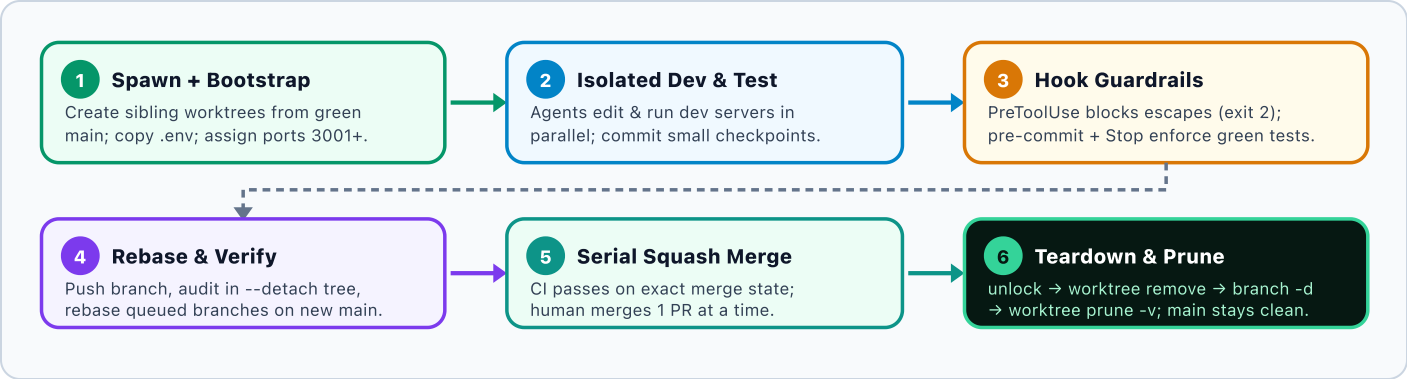
Inside scripts or skills, compare `git rev-parse --git-dir` vs. `--git-common-dir` (and verify `--show-superproject-working-tree` is empty to rule out submodules). If they differ, the agent is *already* inside an isolated worktree—don't nest another!

2. Never Run Bypass Flags on a Credentialed Host

Flags like `--dangerously-skip-permissions`, `--yolo`, and *Run Everything* let a confused or prompt-injected agent overwrite `.git`, delete hooks, and read `~/.ssh`. Reserve them **strictly** for disposable, credential-free containers!

End-to-End Fleet Pipeline: Spawn to Clean Repository

Here is how every piece in this guide locks together into one repeatable engineering pipeline—turning three concurrent tickets into three clean, linear commits on `main` with zero lost work.



Concrete 3-Agent Walkthrough: Shipping Cart Totals, Discounts & Docs Together

Timeline Beat	What Happens Across the Fleet	Safety Mechanism in Action
09:00 • Spawn	Run `wt-spawn.sh` for `wt-totals` (`:3001`), `wt-discounts` (`:3002`), and `wt-docs` (`:3003`) off green `main`.	Shared `.git` spawns all 3 in <10s; `git worktree lock` tags each owner and ticket.
09:22 • Mid-Run	Agent 2 hits a lint error and tries `git commit --no-verify`; Agent 1 runs integration tests against `app_test_wt1`.	`PreToolUse` exits `2`, forcing Agent 2 to fix the lint error properly; DB isolation prevents test crossfire.
09:45 • Merge #1	`wt-totals` passes its `Stop` gate; PR #1 passes CI and squash-merges into `main` (updating `src/index.ts`).	Branch protection requires PR + green status checks; human checkout never left `main`.
09:52 • Rebase #2	Inside `wt-discounts`, run `git fetch && git rebase origin/main`; `src/index.ts` conflicts on barrel exports.	Agent 2 reconciles both export lines inside `wt-discounts`, re-runs `pnpm test`, pushes, and merges cleanly!
10:05 • Cleanup	PR #3 (`wt-docs`) rebases with zero conflicts and merges; all 3 workshops are unlocked, removed, and pruned.	`git worktree list` returns to a single clean `[main]` line with 3 crisp squashed commits in history.

Steady-State Scaling Rule: 3 to 5 Concurrent Workshops per Human Reviewer

Fleet throughput is bounded by human code review at the merge gate—not by how many terminals your laptop can spawn. Three to five parallel workshops is the sweet spot for one engineer with solid CI; beyond that, shrink ticket scope or add a second gatekeeper rather than loosening review!

Troubleshooting Clinic & Fleet Pre-Flight Checklist

Ninety percent of worktree hiccups diagnose themselves with `git worktree list -v` and `git -C <path> status`. Use this table for 10-second recoveries when an agent or folder misbehaves.

Top 7 Worktree & Parallel-Agent Errors (10-Second Fixes)

Error / Symptom	Root Cause & 10-Second Fix
`fatal: 'branch' is already checked out at '...'`	Git's Iron Rule protecting you! Create a distinct branch (`git worktree add ../wt-new -b agent-c/task main`) or pass `--detach` for read-only review. **Never** pass `-f`.
Deleted folder with `rm -rf`; Git still lists it / blocks branch	Orphaned metadata in `.git/worktrees/`. Run `git worktree unlock <path>` (if locked), preview with `git worktree prune -n`, then clear with `git worktree prune -v`.
Ran `mv` on a worktree; Git says `not a git repository`	Moving by hand broke the absolute path contract inside `.git`. From your main repo run `git worktree repair <new-path>` (next time use `git worktree move`).
`Error: listen EADDRINUSE: address already in use :::3000`	Two workshops bound default port `3000`. Write a unique `PORT=3001` / `3002` into each worktree's `.env.local` so dev servers and Playwright runs stay in separate lanes.
`PreToolUse` hook prints an error, but command still runs!	Your script exited with `1` (non-blocking warning). Change the block branch to `exit 2` (or emit JSON `permissionDecision: "deny"`) and verify `chmod +x`.
Husky `pre-commit` fires in `main`, silent in worktrees	Relative `core.hooksPath` (`.husky/_`) is gitignored in fresh checkouts. Run `npm ci` / `pnpm install` inside the worktree so the `prepare` script regenerates shims.
5,000-line `package-lock.json` merge conflict at PR time	Agents ran bare `npm install` concurrently. Standardize on `npm ci` / `pnpm i --frozen-lockfile`; to fix now, merge `package.json`, run `npm install` once, and commit.

6-Point Fleet Pre-Flight Checklist (Verify Before Dispatching Parallel Agents)

- ☐ **1. Green Human `main`:** `git pull --ff-only && npm test` passes before any branch sprouts; humans stay on `main`, agents never do.
- ☐ **2. 1-Agent-1-Worktree-1-Branch:** Spawned via native tool (`--worktree`, `share`, `orca`) or sibling `git worktree add -b` + `lock`.
- ☐ **3. Isolated Runtime Lanes:** Fresh per-worktree `pnpm i` / `.venv`, copied `.env.local`, unique `PORT`, and separate test DB.
- ☐ **4. Active Path & Git Fence:** Committed `PreToolUse` hook returns `exit 2` on cross-folder edits, `reset --hard`, and `--no-verify`.
- ☐ **5. Sandbox On, Bypass Off:** OS sandbox enabled (`workspace-write` / `/sandbox`); zero `--yolo` flags on credentialed hosts.
- ☐ **6. Serial Merge & Prune:** Require up-to-date PR checks, merge 1 PR at a time, rebase remaining workshops, then `remove` + `prune`.